

Instituto Tecnológico y de Estudios Superiores de Occidente

ALGORITMOS Y PROGRAMACIÓN



<Fut-draft>

Presentan

Luis Miguel Quintana Gil 762094

Santiago Morales López 760906

Sergio Abdiel Padilla Mora 764178

Profesor: <Luis Carlos Alvarado Garnica>

Fecha <30 11 2025>

Contenido

[Nombre del proyecto.](#) 3

[Introducción.](#) 3 , 4

[Alcance del Proyecto.](#) 5 , 6

[Requerimientos.](#) 6 , 7, 8

[Diseño de la solución.](#)

[Diagrama de descomposición modular.](#)

[Pseudocódigo.](#)

[Diagrama de flujo.](#)

[Implementación.](#)

[Código Fuente.](#)

[Pruebas.](#)

[Conclusiones.](#)

[Fuentes de información.](#)

[Anexos.](#)

[Glosario](#)

Nombre del proyecto

El nombre de nuestro proyecto es Fut-draft haciendo la semejanza con el modo de juego del Fifa ahora EA FC Ultimate Team

Introducción

En la introducción del proyecto se espera:

- *Dar contexto de la problemática que se atiende.*
- *Abordar el problema que se resuelve.*
- *Exponer la motivación de los integrantes por realizar el proyecto.*
- *Escribir una cuartilla como mínimo y dos como máximo.*
- *Excluir la descripción de conceptos o términos. Si es necesario se recomienda agregarlos al Glosario.*

Introducción

El presente proyecto nace de nuestro interés por el fútbol y los videojuegos, especialmente por el modo FIFA Draft, una de las modalidades más populares del FIFA. En este modo, el jugador arma un equipo eligiendo entre diferentes opciones de jugadores que aparecen de manera aleatoria. El problema es que, para poder jugarlo, se necesitan monedas dentro del juego, lo cual se convierte en una barrera para quienes solo quieren divertirse sin preocuparse por recursos digitales o recompensas.

A partir de esta situación pensamos en crear una versión alternativa del FIFA Draft que sea accesible y gratuita. Queremos conservar la esencia del modo Draft, donde se presentan varias opciones de jugadores por posición y el usuario arma su equipo, pero sin las restricciones del original. Nuestro sistema dará cuatro selecciones por posición, comenzando por el portero, y se repetirá el proceso hasta completar la alineación.

El problema que buscamos resolver es la limitación de acceso al modo Draft original, ya que depende del uso de monedas o puntos de otros modos. Eso genera frustración en muchos jugadores que solo quieren una experiencia rápida y entretenida. Además, en el juego oficial el exceso de azar hace difícil formar equipos competitivos o balanceados. Nuestra propuesta busca equilibrar esto: eliminar el costo de entrada y, al mismo tiempo, mantener la emoción de la elección aleatoria con un control más justo sobre las opciones.

La motivación surge de que nos gusta tanto el fútbol como la programación, y vimos en este proyecto la oportunidad de combinar ambos intereses. Aquí aplicamos lógica, estructuras de control y generación aleatoria para crear una experiencia interactiva. También representa un reto creativo, porque buscamos hacer que la dinámica sea más justa y entretenida.

Más allá del juego en sí, este proyecto también nos hace pensar en la accesibilidad en los videojuegos y cómo los modelos de pago influyen en la experiencia de los

usuarios. Nuestro objetivo no es competir con el FIFA original, sino ofrecer una alternativa sencilla y libre para que cualquiera pueda participar y disfrutar del proceso de armar un equipo.

En conclusión, queremos recrear el espíritu del FIFA Draft con un enfoque más inclusivo y justo. Con este proyecto no solo unimos nuestra pasión por el fútbol y la programación, sino que también aprendemos a diseñar experiencias equilibradas e interactivas que cualquiera pueda disfrutar sin limitaciones.

Alcance del Proyecto

Objetivo General

Desarrollar un programa interactivo que permita al usuario armar un equipo de fútbol a través de un sistema de selección por posiciones, similar a la dinámica del FIFA Draft, donde se presentan diferentes opciones de jugadores para elegir en cada etapa.

Módulos y Funcionalidades

1. Módulo de Inicio

- Pantalla inicial con menú.
- Explicación breve de la dinámica del juego.

2. Módulo de Selección de Jugadores

- Selección del **portero** entre 4 opciones.
- Selección de **defensas** entre 4 opciones.
- Selección de **mediocampistas** entre 4 opciones.
- Selección de **delanteros** entre 4 opciones.

3. Módulo de Formación del Equipo

- Visualización del equipo final seleccionado.
- Posibilidad de mostrar estadísticas básicas de cada jugador (ej. ataque, defensa, velocidad).

4. Módulo de Resultados

- Mostrar el equipo armado al finalizar todas las elecciones.
- Opción de reiniciar para volver a jugar.

Restricciones o Limitaciones

- El número de opciones por posición está limitado a 4 jugadores.
- El programa no simulará partidos, solo la selección de jugadores.
- La base de datos de jugadores será limitada y predefinida, no se incluirá un catálogo completo como en FIFA.
- La interfaz será sencilla y básica (según el lenguaje de programación elegido), sin animaciones avanzadas.
- El alcance se centra en la funcionalidad de selección y armado del equipo, no en la jugabilidad de un partido real.

Requerimientos

Son las necesidades que se requieren cumplir con el programa. Se espera:

1. El sistema debe permitir al usuario iniciar un nuevo draft.
2. El sistema debe generar aleatoriamente 5 opciones de formaciones.
3. El usuario debe poder seleccionar una de las formaciones ofrecidas.
4. El sistema debe mostrar posiciones de los jugadores en el campo según la formación elegida.
5. El sistema debe permitir elegir jugadores en cada posición de la plantilla.

6. El sistema debe tener una base de datos de jugadores (puede ser simulada con JSON o CSV).
7. La base de datos debe incluir atributos como: nombre, posición, club, liga, nacionalidad, valoración y química básica.
8. Al seleccionar un jugador para una posición, el sistema debe mostrar 5 opciones aleatorias de la base de datos.
9. El sistema debe asegurar que las cartas ofrecidas tengan rareza y calidad variadas (oro, plata, bronce; comunes y especiales).
10. El usuario debe poder visualizar estadísticas básicas del jugador antes de elegirlo.
11. El sistema debe permitir cambiar jugadores elegidos en la plantilla antes de finalizar.
12. El sistema debe calcular la química entre jugadores basada en:
 - Nacionalidad
 - Liga
 - Club
13. El sistema debe mostrar visualmente (o con texto) la química individual de cada jugador.
14. El sistema debe calcular la química total del equipo.
15. El sistema debe ofrecer una banca y suplentes (7 jugadores aprox.).

16. El sistema debe permitir seleccionar suplentes con el mismo método de 5 cartas aleatorias.
17. El sistema debe mostrar el 11 inicial, banca, valoración media y química total.
18. El sistema debe calcular un puntaje final del draft (por ejemplo, promedio de valoración + bonificación de química).
19. El usuario debe poder reiniciar el draft en cualquier momento.
20. El sistema debe mostrar de forma clara las opciones de elección (listas o menús).
21. El sistema debe evitar repetir jugadores ya elegidos.
22. El sistema debe incluir una opción para guardar el draft generado en un archivo (ejemplo: JSON o TXT).
23. El sistema debe permitir cargar un draft guardado anteriormente

Diseño de la solución

Aplicando la metodología de resolución de problemas computables y haciendo uso de las técnicas de representación de un algoritmo, es posible a partir del problema planteado, analizar los módulos y funcionalidades del programa, descritos en el alcance del proyecto, y sus requerimientos para diseñar la solución. Por lo tanto, el diagrama de descomposición modular, el pseudocódigo y el diagrama de flujo del algoritmo son parte de esta sección.

primero delimitamos lo que queríamos hacer que fue el draft de fútbol , luego hicimos el diagrama de flujo para ver antes de hacer el código como iba a funcionar , luego el pseudocódigo para ver en palabras el código y ver que podíamos mejorar antes de pasarlo al código y ya al final hicimos el código con librerías y todo lo que pedía en la rúbrica y al final hicimos pruebas para ver que todo funcionaba.

Diagrama de descomposición modular

Representa la descomposición jerárquica del problema en varios módulos y funciones, donde las funciones son el nivel más bajo de la jerarquía. Facilita el diseño de la solución al centrarse en subproblemas más pequeños y simples. En esta sección se espera que el diagrama incluya:

- *Módulos y funcionalidades propios del problema.*
- *Módulos externos utilizados (bibliotecas).*
- *Relación entre módulos.*

Pseudocódigo

Utilizando la técnica de representación textual, diseñar el algoritmo considerando las características de un pseudocódigo: descripción detallada de alto nivel, legible, compacta y especialmente independiente del lenguaje de programación. En este apartado se espera:

- *Representar el módulo principal de la solución.*
- *Describir la solución cumpliendo con el alcance y requerimientos del programa.*
- *Cumplir las reglas de escritura de un pseudocódigo.*
- *Escribir las instrucciones en alguna herramienta computacional (deseable).*
- *Ser coherente con la solución representada gráficamente.*

// Iniciamos la bandera para controlar si el usuario quiere seguir jugando

JugarDeNuevo <- VERDADERO

// Función clave para Data Loading (simulando una carga desde JSON/CSV)

LLAMAR Cargar_Base_Datos_Inicial()

// Módulo introductorio para darle contexto al usuario

LLAMAR Modulo_Inicio()

// Bucle de Ejecución del Juego: Es nuestro "while(true)" con una condición de salida controlada

MIENTRAS JugarDeNuevo ES VERDADERO HACER

// Reseteamos el estado del equipo y le damos opciones de formación

LLAMAR Iniciar_Nuevo_Draft()

// Aquí ocurre el grueso del juego: pick de jugadores por posición

LLAMAR Modulo_Seleccion_Jugadores()

// Calculamos las métricas de Ciencia de Datos: Promedios de Valoración y Química

LLAMAR Modulo_Resultados()

// Control de flujo para el reinicio

Escribir "¿Se echa otro draft? (S/N)"

Leer OpcionReiniciar

SI OpcionReiniciar ES IGUAL A "S" O OpcionReiniciar ES IGUAL A "s" ENTONCES

JugarDeNuevo <- VERDADERO

SINO

```

    JugarDeNuevo <- FALSO // Rompemos el ciclo principal

    FIN_SI

FIN_MIENTRAS

Escribir "Programa finalizado. ¡Nos vemos en el próximo proyecto!"

FIN // DEL ALGORITMO PRINCIPAL

//
=====
===== // FUNCIONES Y MÓDULOS (Los módulos más detallados de la solución) //
=====
=====

FUNCION Cargar_Base_Datos_Inicial() // Simula la lectura del set de datos inicial
(nombres, clubes, atributos) Escribir "Inicializando el set de datos de jugadores..." FIN
FUNCION

FUNCION Modulo_Inicio() // Muestra la interfaz de bienvenida y las instrucciones rápidas
Escribir "--- BIENVENIDO A FUT-DRAFT SIMULATOR v1.0 ---" Escribir "Instrucción clave:
Elige la carta que maximice tu valoración y química." ESPERAR_ENTRADA() FIN
FUNCION

FUNCION Iniciar_Nuevo_Draft() // Setup rápido para empezar de cero

    // Limpiamos las listas para evitar arrastrar jugadores del draft anterior

    EquipoUsuario <- LISTA VACIA

    BancaSuplentes <- LISTA VACIA

    JugadoresElegidos <- LISTA VACIA

    // Generar 5 formaciones aleatorias (usando una función de Random)

    OpcionesFormacion <- GENERAR_ALEATORIO_FORMACIONES(5)

    // Interacción para elegir la formación

```

Escribir "Seleccione una de las 5 formaciones generadas (Ej: 4-3-3):"

// Mostrar lista de OpcionesFormacion

Leer IndiceFormacion

FormacionElegida <- OpcionesFormacion[IndiceFormacion]

Escribir "¡Formación " + FormacionElegida + " cargada!"

Posiciones11Inicial <-

OBTENER_POSICIONES_POR_FORMACION(FormacionElegida)

Escribir "--- FASE I: ARMANDO EL 11 TITULAR ---"

// Bucle para cubrir todas las posiciones del esquema (POR, DEFs, MEDs, DELs)

PARA Posicion EN Posiciones11Inicial HACER

LLAMAR Seleccionar_Jugador_Para(Posicion, EquipoUsuario)

FIN_PARA

Escribir "--- FASE II: SELECCIÓN DE BANCA (7 PUESTOS) ---"

// 7 iteraciones para la banca (sin restricción estricta de posición)

PARA i DESDE 1 HASTA 7 HACER

LLAMAR Seleccionar_Jugador_Para("SUPLENTE", BancaSuplentes)

FIN_PARA

// Oportunidad final de optimización

LLAMAR Gestionar_Cambios_Finales(EquipoUsuario, BancaSuplentes)

// Filtra la DB por posición y asegura no duplicar jugadores (eficiencia de búsqueda)

```
OpcionesAleatorias <- GENERAR_CARTAS_ALEATORIAS(Posicion, 5,  
BaseDatosJugadores, JugadoresElegidos)
```

```
// Muestra las 5 opciones con sus stats clave (OVR, Club, Liga)
```

```
Escribir "Turno para: " + Posicion + ". Elija entre las 5 opciones:"
```

```
// Bucle para Escribir cada Jugador en OpcionesAleatorias
```

```
Leer SeleccionUsuario
```

```
JugadorElegido <- OpcionesAleatorias[SeleccionUsuario]
```

```
// Actualiza el equipo y la lista de jugadores ya usados
```

```
AGREGAR JugadorElegido A Plantilla
```

```
AGREGAR JugadorElegido.Nombre A JugadoresElegidos
```

```
// Recalcula la Química porque el nuevo jugador afecta al resto
```

```
LLAMAR Calcular_Quimica_Individual(Plantilla)
```

```
PARA Jugador EN PlantillaCompleta HACER
```

```
    // Lógica de cálculo basada en Club, Liga y Nacionalidad para generar un score
```

```
    PuntuacionQuimica <- CALCULAR_QUIMICA_DEL_JUGADOR(Jugador,  
    PlantillaCompleta)
```

```
    Jugador.QUIMICA_INDIVIDUAL <- PuntuacionQuimica // Actualizamos el atributo
```

```
FIN_PARA
```

```
Escribir "--- RESULTADOS FINALES Y MÉTRICAS CLAVE ---"
```

```
// Agregación de datos: cálculo de promedios para las métricas de éxito
```

```
ValoracionMedia <- CALCULAR_MEDIA_VALORACION(EquipoUsuario)
```

```
QuimicaTotal <- CALCULAR_MEDIA_QUIMICA(EquipoUsuario)
```

```
// Output del equipo completo
```

```
Escribir "Esquema usado: " + FormacionElegida
```

```
Escribir "11 TITULAR (El que cuenta para el score):"
```

```
// Escribir lista de EquipoUsuario
```

```
Escribir "7 SUPLENTE:"
```

```
// Escribir lista de BancaSuplentes
```

```
Escribir "MÉTRICA 1 (OVR Promedio): " + ValoracionMedia
```

```
Escribir "MÉTRICA 2 (Química Promedio): " + QuimicaTotal
```

```
// Cálculo final basado en la ponderación de las dos métricas
```

```
PuntajeFinal <- CALCULAR_PUNTAJE_FINAL(ValoracionMedia, QuimicaTotal)
```

```
Escribir "SCORE FINAL DEL DRAFT: " + PuntajeFinal + " (¡Bien hecho!)"
```

```
// Persistencia de datos (feature de DS, claro)
```

```
Escribir "¿Guardamos este gran draft en un archivo? (S/N)"
```

```
Leer OpcionGuardar
```

```
SI OpcionGuardar ES IGUAL A "S" ENTONCES
```

```
    LLAMAR Guardar_Draft(EquipoUsuario, BancaSuplentes, FormacionElegida,  
        PuntajeFinal)
```

```
FIN_SI
```

```
FIN FUNCION
```

Diagrama de flujo

Utilizando la técnica de representación gráfica, diseñar el algoritmo considerando las características de un pseudocódigo: descripción detallada de alto nivel, legible, compacta y especialmente independiente del lenguaje de programación. En este apartado se espera:

- *Representar el módulo principal de la solución*
- *Describir la solución cumpliendo con el alcance y requerimientos del programa*
- *Cumplir las reglas de construcción de un diagrama de flujo.*
- *Escribir las instrucciones en alguna herramienta computacional (deseable)*
- *Ser coherente con la solución representada textualmente.*

Sergio subió el diagrama de flujo aparte para que no se viera todo feo en el documento

<https://drive.google.com/file/d/1IXD0t0YRD7aQZ-VY5yPikoQ5IYsOshoG/view?usp=sharing>

Implementación

Una vez definida la solución mediante el análisis del problema y el diseño del algoritmo, se procede a planificar su implementación en el lenguaje de programación Python. La implementación se estructurará de forma modular para facilitar el mantenimiento, la lectura del código y la ejecución de pruebas unitarias e integrales.

El programa estará dividido en varios módulos, cada uno encargado de una tarea específica:

1. Módulo de carga de datos

Este módulo se encargará de leer la información de los jugadores desde un archivo de Excel.

Se utilizará la librería pandas para importar el archivo y convertirlo en una estructura manejable dentro del programa (por ejemplo, una lista de diccionarios).

2. Módulo de filtrado de jugadores

Aquí se implementará una función encargada de obtener los jugadores disponibles según la posición requerida.

También verificará que los jugadores ya seleccionados no vuelvan a ser elegidos, evitando repeticiones.

3. Módulo de selección aleatoria

Para simular la dinámica de un “FUT Draft”, se implementará una función que limite las opciones del usuario a un máximo de cuatro jugadores seleccionados de forma aleatoria. Esto requerirá el uso de la librería estándar **random**.

4. Módulo de interacción con el usuario

Este módulo mostrará las opciones disponibles y permitirá al usuario seleccionar un jugador mediante entrada por teclado.

Se validará que el usuario ingrese valores numéricos dentro del rango permitido, manejando errores y entradas incorrectas.

5. Módulo de construcción del equipo

En este módulo se integrarán los módulos anteriores.

El sistema recorrerá una lista predefinida de posiciones requeridas y, para cada una, mostrará al usuario cuatro opciones posibles.

Los jugadores seleccionados se irán almacenando en una estructura de datos que representará al equipo final.

6. Módulo de cálculo de química y valoración

Una vez completado el equipo, se implementará un módulo encargado de calcular:

- La valoración promedio del equipo.
- La química individual y total, basada en criterios como nacionalidad, liga y club.
- Un puntaje final que combine ambos indicadores.

Para ello se utilizarán estructuras de control como ciclos y condicionales.

7. Módulo de presentación de resultados

Finalmente, se implementará un módulo encargado de mostrar de manera ordenada el equipo creado, junto con:

- La posición de cada jugador.
- Su nombre y valoración.
- Su química individual.
- Las estadísticas globales del equipo

Código Fuente

```
#instruccion: Crear un codigo que imite la mecanica del futdraft en
    el videojuego FIFA usando python

# necesitamos usar librerias, todos tipo de ciclos y que se cargue
    desde un archivo que funja como base de datos

import pandas as pd

import random


# 1. CARGA EL ARCHIVO EXCEL DE LOS JUGADORES PARA EL FUTDRAFT


def cargar_jugadores_excel(nombre_archivo):

    df = pd.read_excel(nombre_archivo)

    return df.to_dict(orient="records")


# 2. OBTENER JUGADORES DISPONIBLES POR POSICIÓN


def obtener_disponibles(jugadores, posicion, elegidos):

    disponibles = [

        j for j in jugadores

        if str(j["Posición"]).upper() == posicion and j["Nombre"]
        not in elegidos

    ]

    if len(disponibles) == 0:

        disponibles = [j for j in jugadores if j["Nombre"] not in
            elegidos]
```

```
        return disponibles

# 3. REDUCIR A SOLO 4 OPCIONES RANDOM

def seleccionar_4_random(disponibles):

    if len(disponibles) <= 4:

        return disponibles

    return random.sample(disponibles, 4)

# 4. MOSTRAR OPCIONES

def mostrar_opciones(disponibles):

    for i, jugador in enumerate(disponibles):

        print(f"{i+1}. {jugador['Nombre']} | VAL: {jugador['Valoración']} | CLUB: {jugador['Club']}")

# 5. PEDIR ELECCIÓN AL USUARIO

def seleccionar_jugador(disponibles):

    while True:

        entrada = input(f"Elige jugador (1-{len(disponibles)}): ")
        entrada = entrada.strip()

        if not entrada.isdigit():
```

```
        print("Entrada no es un número. Intenta de nuevo.")

        continue

    idx = int(entrada) - 1

    if 0 <= idx < len(disponibles):

        return disponibles[idx]

    else:

        print("Número fuera de rango. Intenta de nuevo.")


# 6. CONSTRUIR EQUIPO

def construir_equipo(jugadores, posiciones):

    equipo = []

    elegidos = []

    print("Seleccionarás jugadores para las siguientes\nposiciones:\n")

    print(posiciones)

    for posicion in posiciones:

        print(f"\nSelecciona jugador para posición: {posicion}")

        disponibles = obtener_disponibles(jugadores, posicion,
elegidos)
```

```

        if len(disponibles) == 0:

            print("No quedan jugadores disponibles.")

            break

        # → esto es para que se limite a los 4 jugadores random y no
        te de a todos los que estan disponibles

        # en esa posición

        disponibles = seleccionar_4_random(disponibles)

        mostrar_opciones(disponibles)

        elegido = seleccionar_jugador(disponibles)

        equipo.append(elegido)

        elegidos.append(elegido["Nombre"])

    return equipo

# 7. CÁLCULO DE LA QUÍMICA Y LA MEDIA DEL EQUIPO

def calcular_quimica_y_media(equipo):

    total_val = 0

    total_qui = 0

    for jugador in equipo:

```

```
try:
    val_num = float(jugador.get("Valoración", 0))

except:
    val_num = 0.0

total_val += val_num

quimica = 0

for compa in equipo:
    if compa is not jugador:
        if jugador["Nacionalidad"] == compa["Nacionalidad"]:
            quimica += 1

        if jugador["Liga"] == compa["Liga"]:
            quimica += 1

        if jugador["Club"] == compa["Club"]:
            quimica += 1

jugador["quimica_individual"] = quimica
total_qui += quimica

prom_val = total_val / len(equipo) if equipo else 0
puntaje_final = prom_val + (total_qui * 0.5)

return prom_val, total_qui, puntaje_final
```

```
# 8. MOSTRAR EL EQUIPO FINAL.
```

```
def mostrar_resultados(equipo, prom_val, total_qui, puntaje_final):  
  
    print("\n=====")  
  
    print("          RESULTADOS DEL EQUIPO")  
  
    print("=====")  
  
    if equipo:  
  
        for j in equipo:  
  
            print(f"{j['Posición']}: {j['Nombre']} | VAL:  
{j['Valoración']} | QUI: {j['quimica_individual']}")  
  
    else:  
  
        print("No se seleccionó ningún jugador.")  
  
  
    print("\n--- ESTADÍSTICAS ---")  
  
    print(f"Valoración media: {prom_val:.2f}")  
  
    print(f"Química total: {total_qui}")  
  
    print(f"Puntaje final: {puntaje_final:.2f}")
```

```
# 9. MAIN (SIRVE PARA QUE SE VAYAN EJECUTANDO LAS FUNCIONES CUANDO  
VAYAN SIENDO NECESITADAS)
```

```
def main():
```

```

    POSICIONES_DRAFT = ["POR", "DFC", "DFC", "LTI", "LTD", "MCD",
    "MC", "MCO", "EI", "ED", "DC"]

    print("=====")

    print(" FUT DRAFT ")

    print("=====\\n")

    jugadores = cargar_jugadores_excel("jugadores para
    futdraft.xlsx")

    equipo = construir_equipo(jugadores, POSICIONES_DRAFT)

    prom_val, total_qui, puntaje_final =
    calcular_quimica_y_media(equipo)

    mostrar_resultados(equipo, prom_val, total_qui, puntaje_final)

if __name__ == "__main__":

    main()

```

Pruebas

En las pruebas del programa se espera: FF

- Probar las funcionalidades de cada módulo.
- Realizar pruebas de tipo unitarias e integrales.
- Presentar al menos 3 pruebas por cada módulo.
- Presentar imágenes de resultados, agrupadas por módulo.

prueba 1:

```
1. Sommer | VAL: 85 | CLUB: INT
2. Ramsdale | VAL: 84 | CLUB: ARS
3. Ter Stegen | VAL: 89 | CLUB: BAR
4. Courtois | VAL: 90 | CLUB: RMA
```

aquí podemos ver que te da jugadores todos dentro de la misma posición, solo te da 4 y te da aleatorios

```
Selecciona jugador para posición: EI
1. Rebic | VAL: 82 | CLUB: MIL
2. Foden | VAL: 86 | CLUB: MCI
3. Boga | VAL: 81 | CLUB: NIZ
4. Luis Díaz | VAL: 86 | CLUB: LIV
Elige jugador (1-4):
Entrada no es un número. Intenta de nuevo.
Elige jugador (1-4): █
```

Aquí podemos comprobar que si llegas a poner un número que no sea el correcto, el código no se congela y sigue su curso pidiéndote que introduzcas un número correcto

```
--- ESTADÍSTICAS ---
Valoración media: 84.45
Química total: 62
Puntaje final: 115.45
```

aquí vemos que los cálculos de química media y el puntaje final se están haciendo correctamente

```
=====
                        RESULTADOS DEL EQUIPO
=====
POR: Ramsdale | VAL: 84 | QUI: 8
DFC: Pepe | VAL: 82 | QUI: 0
DFC: Kounde | VAL: 86 | QUI: 3
LTI: Chilwell | VAL: 84 | QUI: 8
LTD: Arnau Martínez | VAL: 82 | QUI: 3
MCD: Fabinho | VAL: 85 | QUI: 7
MC: Gavi | VAL: 85 | QUI: 4
MCO: De Bruyne | VAL: 91 | QUI: 7
EI: Foden | VAL: 86 | QUI: 9
ED: Mbeumo | VAL: 82 | QUI: 6
DC: Matheus Cunha | VAL: 82 | QUI: 7
```

aquí vemos como nos entrega el equipo con todas las características de cada jugador

en el resto de pruebas veremos como todas las funciones siguen haciendo su trabajo correctamente y como hay aleatoriedad

prueba 2

```
Selecciona jugador para posición: POR
1. Navas | VAL: 85 | CLUB: PSG
2. Donnarumma | VAL: 89 | CLUB: PSG
3. Neuer | VAL: 88 | CLUB: BAY
4. Oblak | VAL: 90 | CLUB: ATM
Elige jugador (1-4):
```

```
--- ESTADÍSTICAS ---
Valoración media: 85.45
Química total: 36
Puntaje final: 103.45
```

```
=====
                        RESULTADOS DEL EQUIPO
=====
POR: Donnarumma | VAL: 89 | QUI: 2
DFC: Hummels | VAL: 84 | QUI: 3
DFC: Kounde | VAL: 86 | QUI: 6
LTI: Theo Hernández | VAL: 86 | QUI: 5
LTD: Walker | VAL: 85 | QUI: 1
MCD: Casemiro | VAL: 89 | QUI: 1
MC: Vitinha | VAL: 83 | QUI: 2
MCO: Reus | VAL: 85 | QUI: 3
EI: Diaby | VAL: 84 | QUI: 5
ED: Yamal | VAL: 85 | QUI: 3
DC: Giroud | VAL: 84 | QUI: 5
```

prueba 3

```
Selecciona jugador para posición: LTI
1. Robertson | VAL: 87 | CLUB: LIV
2. Chilwell | VAL: 84 | CLUB: CHE
3. Guerreiro | VAL: 84 | CLUB: BAY
4. Theo Hernández | VAL: 86 | CLUB: MIL
Elige jugador (1-4): █
```

```
--- ESTADÍSTICAS ---
Valoración media: 85.73
Química total: 50
Puntaje final: 110.73
```

=====			
RESULTADOS DEL EQUIPO			
=====			
POR:	Courtois	VAL: 90	QUI: 3
DFC:	Ruben Dias	VAL: 88	QUI: 7
DFC:	Varane	VAL: 86	QUI: 6
LTI:	Chilwell	VAL: 84	QUI: 6
LTD:	Walker	VAL: 85	QUI: 8
MCD:	Rodri	VAL: 89	QUI: 7
MC:	Onana	VAL: 82	QUI: 6
MCO:	Griezmann	VAL: 88	QUI: 3
EI:	Chiesa	VAL: 85	QUI: 1
ED:	Pulisic	VAL: 82	QUI: 1
DC:	Dovbyk	VAL: 84	QUI: 2

Conclusiones

La reflexión acerca de los aprendizajes adquiridos, los problemas enfrentados, los hallazgos, retos y desafíos deben ser parte del proyecto, por lo que se espera:

- *Dar una conclusión individual. Cada integrante del equipo deberá expresar su pensar y sentir respecto al proyecto desarrollado, sea en términos del proceso, resultados y demás.*
- *Dar una conclusión grupal. Como equipo expresar su experiencia en el trabajo colaborativo del proyecto y demás.*

Luis Miguel Quintana Gil

Mi conclusión personal después de estar el semestre completo realizando este proyecto es que aquí aplique perfectamente todos los conocimientos que adquirí durante todo el año, y la verdad fue algo que se sintió bastante genuino mientras los ibas haciendo, ya que con cada nuevo tema que íbamos viendo en clase surgen las ideas de cómo poder implementar estos conceptos para que pudiéramos darle forma al proyecto, comparar las diferentes versiones de nuestro código me muestra lo mucho que aprendimos y lo más satisfactorio es que al final el resultado fue lo que esperábamos, la verdad fue un gran proyecto y me divertí mucho haciendolo.

Santiago Morales Lopez

El desarrollo de este simulador fue una experiencia altamente satisfactoria y formativa, centrada en la aplicación práctica de la lógica de programación sobre datos reales.

Mi principal aprendizaje se concentró en la manipulación y gestión de datos externos mediante el uso de librerías especializadas como Pandas. Entendí que la lectura y la posterior limpieza de los tipos de dato son pasos obligatorios para asegurar que el motor central del programa reciba información utilizable (números para las valoraciones y cadenas de texto estandarizadas para las posiciones).

A nivel de la lógica, el desafío fue dominar las estructuras de filtrado avanzadas. Logré implementar un sistema que aplica dos reglas simultáneas a toda la base de datos: el requisito de la posición y la condición de la unicidad (que el jugador no haya sido seleccionado antes). Esta lógica se optimizó con estructuras concisas que hacen el código más eficiente y legible.

Estoy muy complacido con el resultado funcional del Draft. El cálculo final, que combina la valoración media con la química total, valida el éxito de la aplicación de todas las fórmulas de juego. Mi reto futuro es trasladar este núcleo lógico a una interfaz gráfica que mejore la experiencia del usuario.

Sergio Abdiel Padilla Mora

Mi conclusión personal sobre este proyecto es que realmente me ayudó a entender cómo todos los temas que vimos durante el semestre se conectan entre sí y toman sentido cuando los aplicas a algo real. Desde el inicio, la idea de recrear un modo tipo FUT Draft se veía sencilla en teoría, pero conforme avanzamos me di cuenta de que implicaba mucha más lógica, organización y precisión de la que imaginaba.

Uno de mis aprendizajes más grandes fue entender la importancia de estructurar bien un programa antes de empezar a programarlo. Entre el diseño del pseudocódigo, el diagrama de flujo y la modularización, noté cómo cada parte del proceso facilita la siguiente. A veces creía que algún módulo ya estaba listo y al final descubríamos errores lógicos o limitaciones que no habíamos previsto, así que tuve que adaptarme, corregir y pensar en soluciones más eficientes. Eso me enseñó que programar es más pensar que escribir código.

También enfrenté varios retos, especialmente al trabajar con datos externos y al intentar que el simulador fuera realmente dinámico y aleatorio. Hubo momentos en los que las funciones no regresaban lo que esperábamos, cuando se repetían jugadores, o cuando el cálculo de química no coincidía con lo planeado. Pero esos mismos problemas fueron los que más me hicieron aprender: cada falla que aparecía me obligaba a entender mejor qué estaba pasando detrás del código.

Algo que me dejó un gran aprendizaje fue el trabajo en equipo. Ver cómo cada uno aportó desde diferentes áreas y cómo nuestro código fue evolucionando con ideas nuevas, mejoras y correcciones, me ayudó a valorar más la comunicación y la organización.

Comparar la primera versión del proyecto con la final me hace ver lo mucho que crecimos durante el proceso.

Al final, estoy satisfecho con el resultado porque siento que este proyecto realmente reflejó lo que aprendimos en Algoritmos y Programación: manejar datos, crear funciones, controlar flujos, generar aleatoriedad y estructurar un programa funcional desde cero. Me divertí, me frustré, aprendí y crecí, y eso es lo que me quedo como experiencia personal y académica.

Conclusión general: como equipo, el proyecto demostró la eficacia de la modularización del código. La división de responsabilidades por módulos (Carga de Datos, Lógica de Filtros y Motor de Puntuación) permitió que el desarrollo avanzara de manera paralela, logrando una integración exitosa.

El mayor logro técnico grupal fue la implementación del motor de cálculo de Química. Diseñar una estructura de iteración que permitiera comparar a cada jugador con todos sus compañeros y asignar puntos por coincidencias de Nacionalidad, Liga o Club, fue crucial para obtener el puntaje final y demostró el dominio de las estructuras de control de flujo.

Esta experiencia confirmó que el factor más crítico en el trabajo en equipo es la claridad en la entrega de datos entre módulos. Asegurar que cada función recibiera y devolviera el tipo de dato exacto que la siguiente función esperaba (por ejemplo, la lista de diccionarios) fue el pilar para evitar fallos y garantizar un sistema coherente.

Fuentes de información

En las fuentes de información se espera incluir:

- *Referencias bibliográficas, sitios web, tutoriales, etc.*
- *Créditos a recursos utilizados.*

OpenAI. (2025). ChatGPT (versión 5.1) [Modelo de lenguaje]. <https://chat.openai.com/>

nos ayudó a hacer la lista de jugadores con sus nacionalidades, liga , club , posición y valoración.

Anexos

En esta sección se espera información adicional que no clasifica en ninguno de los puntos anteriores.

- *Incluir el modelo de datos (tablas de archivos tabulares, archivos de texto).*
- *Complementar con procesos de negocio u operación.*
- *Incorporar imágenes que den soporte al proyecto.*

El proyecto utiliza un modelo de datos basado en un archivo tabular en formato XLSX/CSV, denominado jugadores para futdraft.xlsx. El archivo contiene las siguientes columnas:

- Nombre: Nombre del jugador.
- Posición: Posición en el campo (ej. DC, ED, MCO).
- Club: Equipo al que pertenece el jugador.
- Liga: Liga en la que juega el club.
- Nacionalidad: País de origen del jugador.
- Valoración: Rating o calificación general del jugador.

Este modelo de datos sirve como base para la selección, filtrado y construcción del futdraft. El archivo fue generado con apoyo de la herramienta de inteligencia artificial ChatGPT (OpenAI, 2025).

Glosario

Lista de términos y su descripción. Se espera:

- *Incluir los términos en orden alfabético.*